Aula 6 – Heaps

Inserção

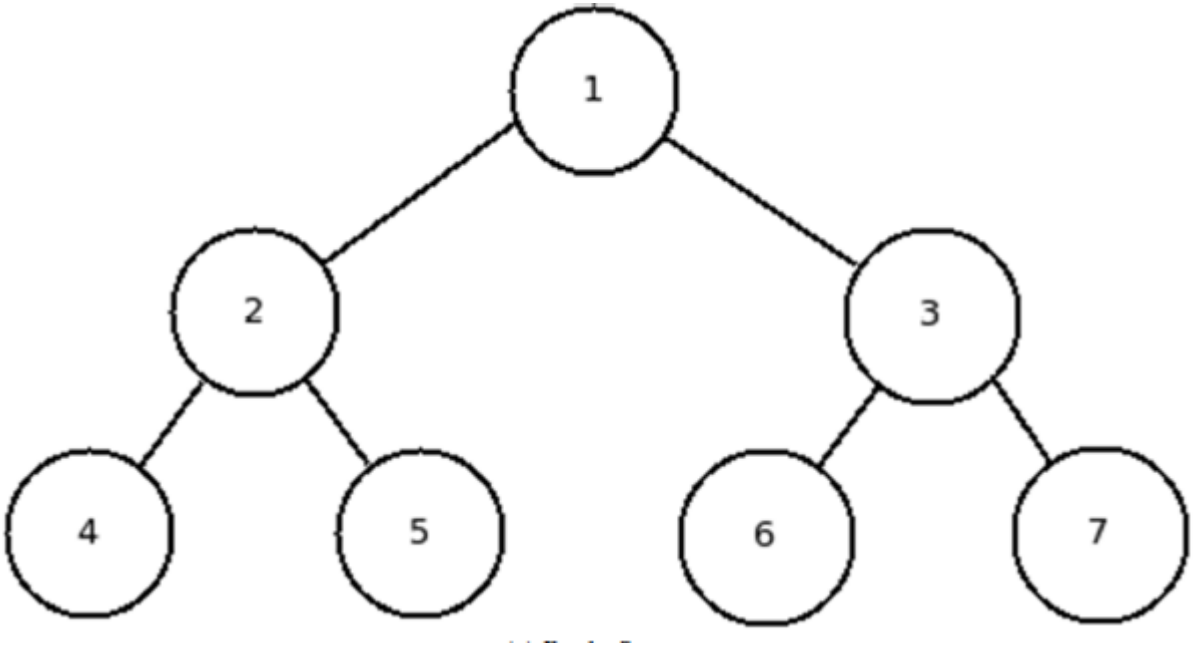
Pesquisa

Eliminação

Exemplos

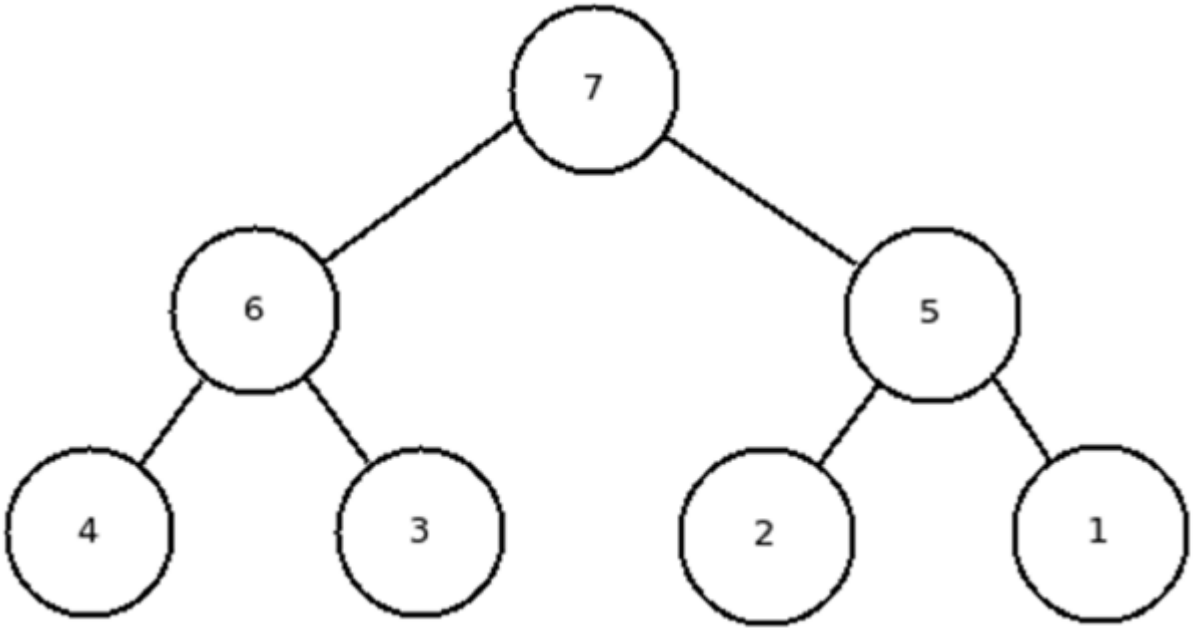
Heaps

Min Heap: Cada nodo tem que se **menor** que os seus descendentes



Heaps

Max Heap: Cada nodo tem que se **maior** que os seus descendentes



Heaps

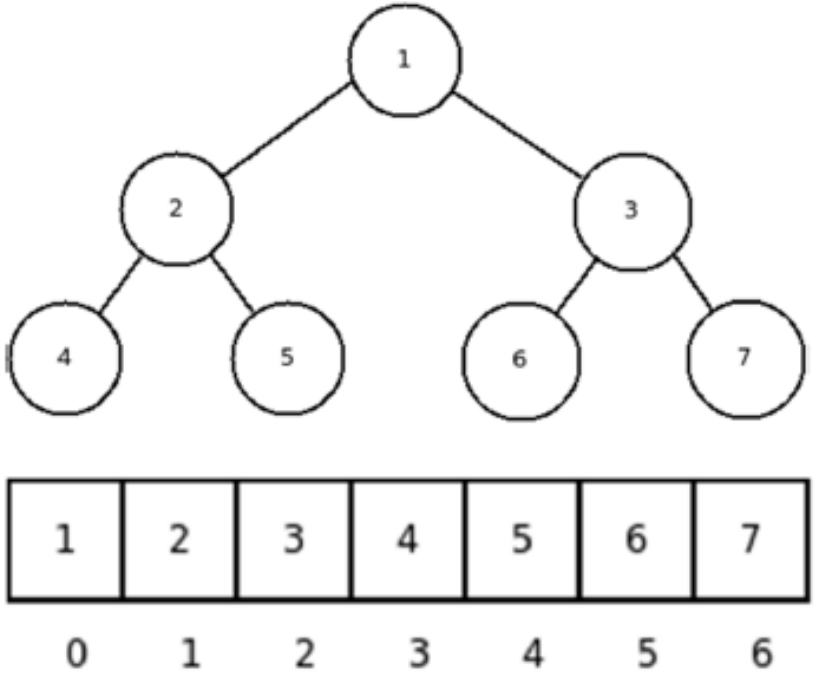
Inserção O(logn) - Remoção O(logn) - Pesquisa O(1) ou O(N)

Métodos:

- void add(Object o); // adiciona um objecto na árvore
- boolean remove(Object o); // remove um objecto
- boolean contains(Object o); // se contem um objecto
- Object peek(Object o); // devolve o primeiro elemento

Min Heap

Os heaps são árvores representadas sobre arrays:



- Ascedente de um nodo:
  - (index-1)/2
- Descedentes de um nodo:
  - 2*index+1
  - 2*index+2

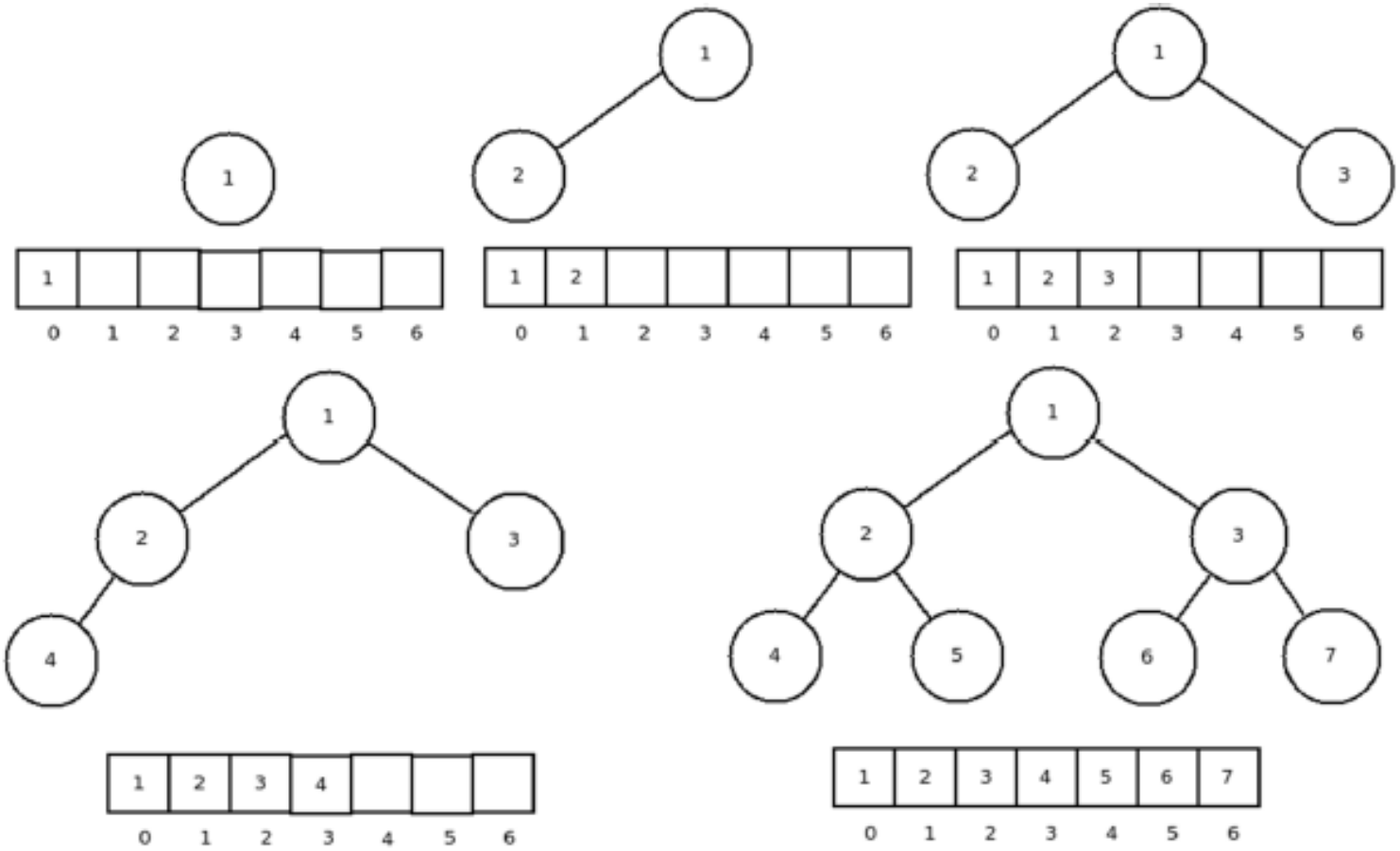
Min Heap

```
Heap {
    Object array[]
    int count=0;

    int left(int index) {
       return 2*index+1;
    }
    int rigth(int index) {
        return 2*index+2;
    }
    int parent(int index) {
        return (index-1)/2;
    }
}
```

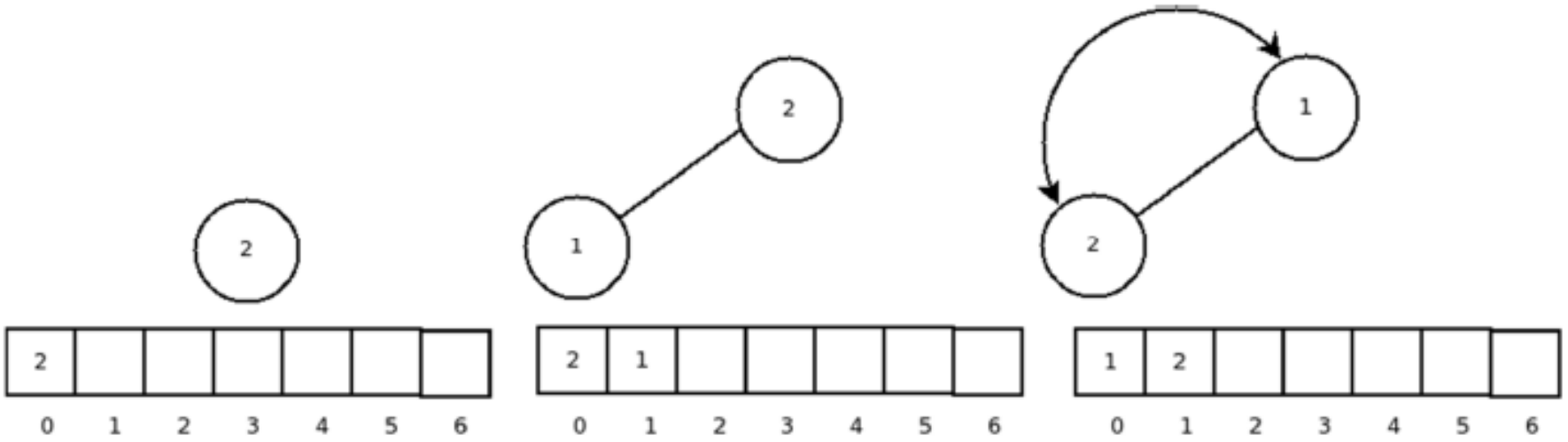
Min Heap: Inserção

1. Inserir na primeira posição livre.



Min Heap: Inserção

2. Reorganizar o Heap no final de cada inserção.



Min Heap: Inserção

```
void add(Object o) {
    array[count] = o
    count = count + 1
    reorganiza()
}

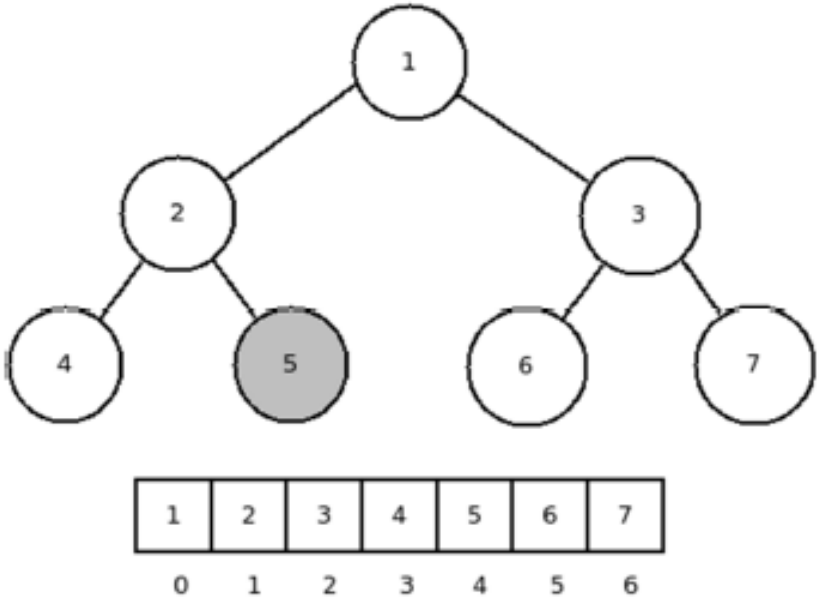
 void reorganiza() {
     i = count -1
     Enquanto i > 0 e array[i] < array[(i-1)/2]
         troca(i, (i-1)/2)
         i = (i-1)/2
 }
```

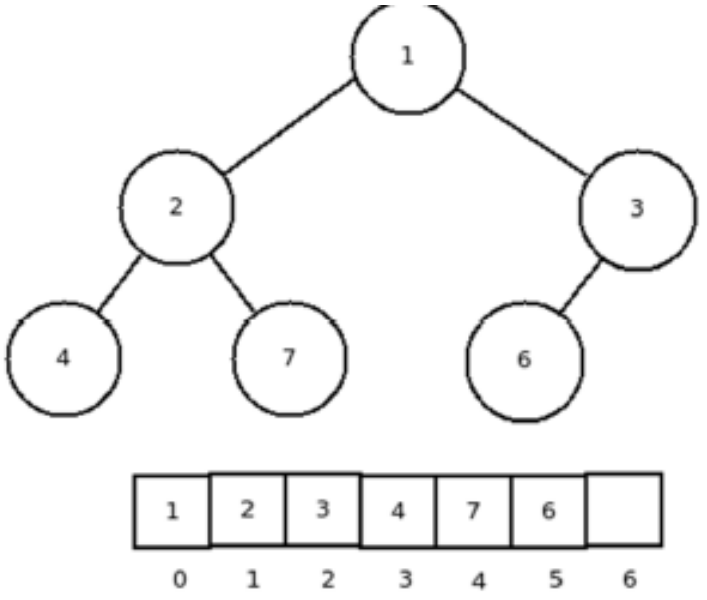
Min Heap: Remoção

1. Procurar o index do item a remover

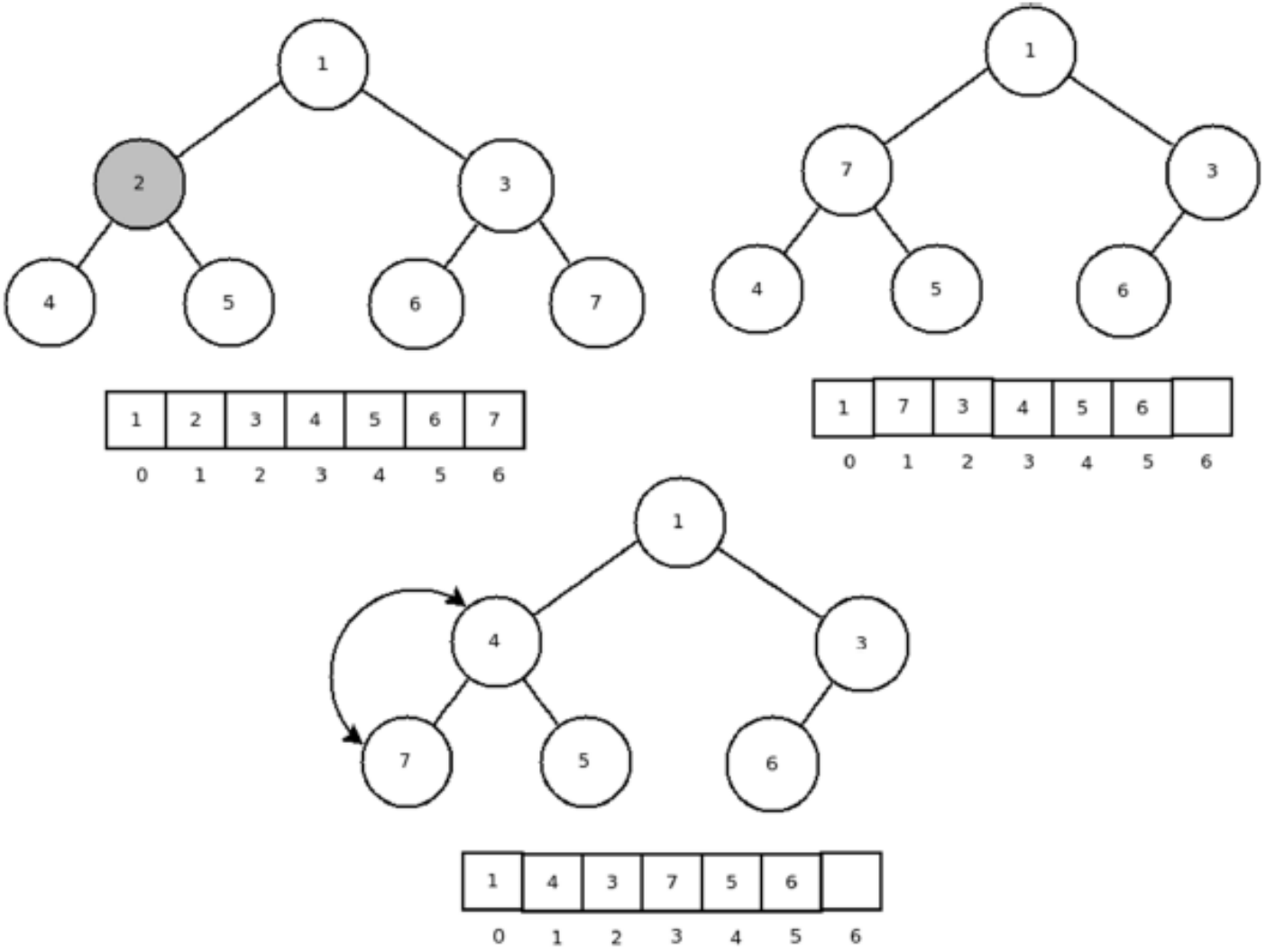
2. Colocar o último item do heap no sitio do item a remover

3. Reorganizar as sub-arvores do item que foi removido. Promove o menor item das sub-arvores, se o valor que foi para a posição do item removido for maior que o seu pai. Se for menor faz a reorganização para cima como na inserção

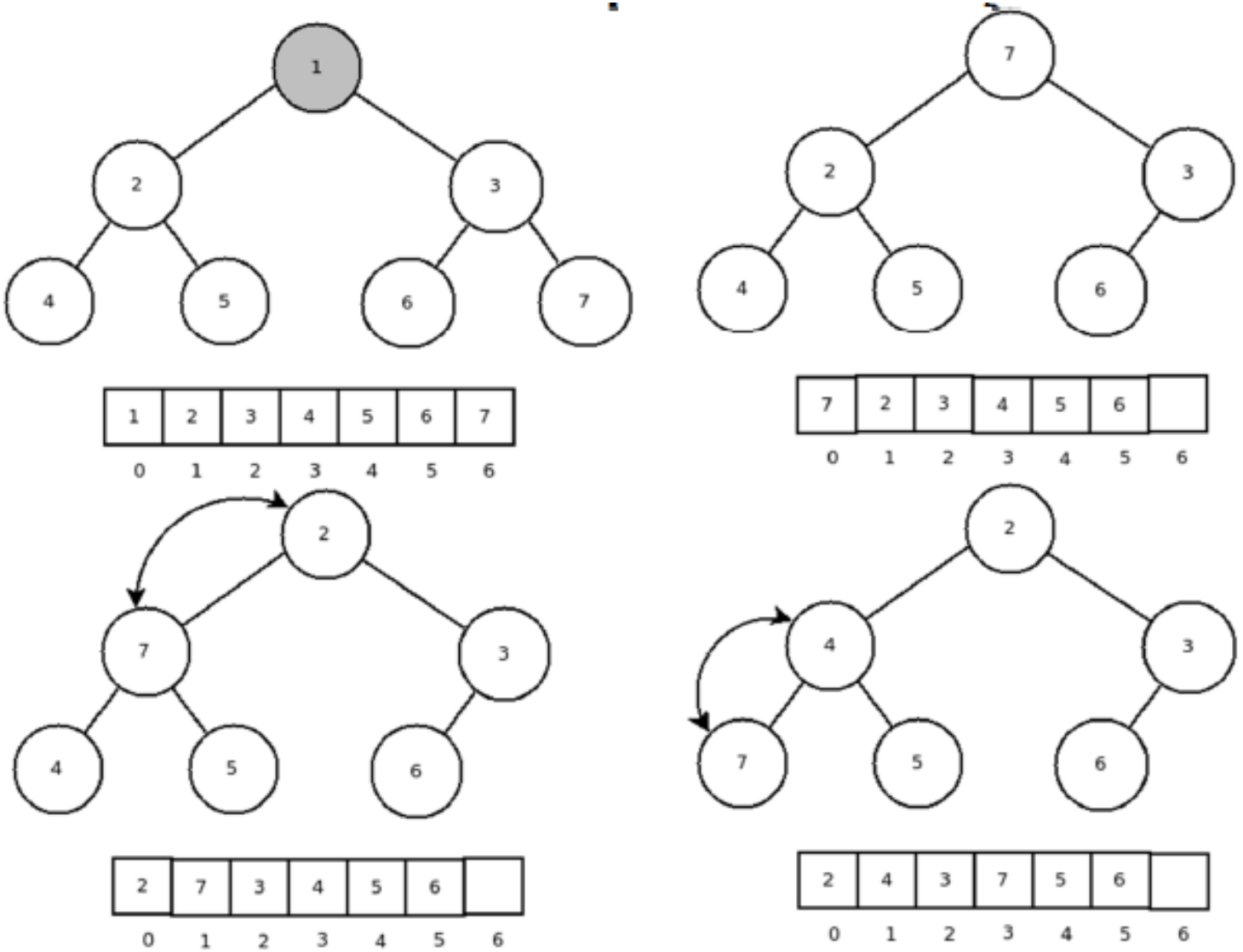




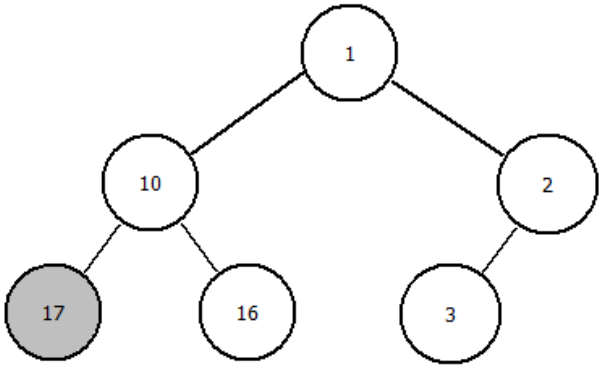
Min Heap: Remoção

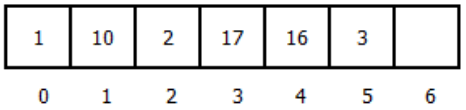


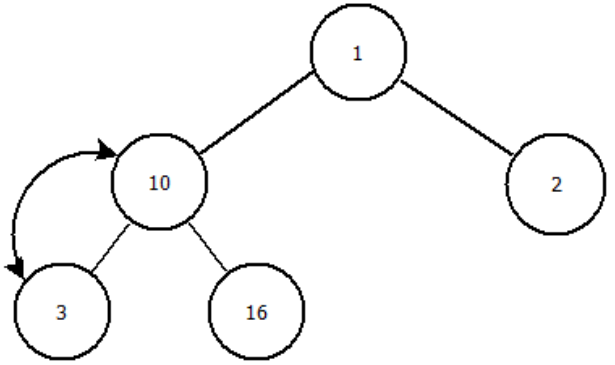
Min Heap: Remoção

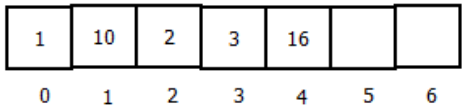


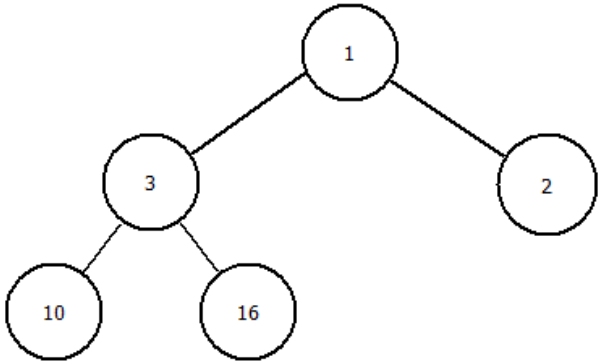
Min Heap: Remoção

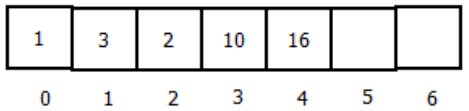












Min Heap: Remoção

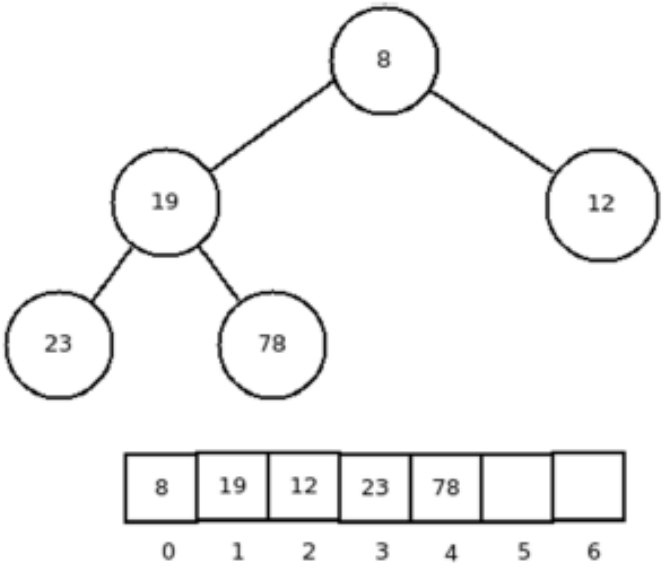
```
boolean remove(Object o) {
    index = findIndex(o)
    if index < 0
        devolve falso
    senão
        count = count – 1
        array[index] = array[count]
        se array[index] < array[parent(index)]
          enquanto index > 0 e array[index] < array[(index-1)/2]
            troca(index, (index-1)/2)
            index = (index-1)/2
        else
          enquanto left(index) < count e
                        ( array[index] > array[left(index)] ou array[index] > array[rigth(index)] )
            se array[left(index)] < array[rigth(index]
                troca(left(index), index)
                index = left(index)
            senão
                troca(rigth(index), index)
                index = rigth(index)
        devolve verdadeiro
}
```

Min Heap: Pesquisa

```
boolean contains(Object o) {
    i = 0
    Enquanto i < count e array[i] <> o
        i = i + 1
    Se i < count
        devolve verdadeiro
    senão
        devolve falso
}
```

Min Heap: Pesquisa Optimizada

```
boolean contains(Object o) {
    inicio = 0
    nodos = 1
    enquanto inicio < count
        inicio = nodos -1
        fim = nodos + inicio
        numNodos = 0
        enquanto inicio < count e inicio < fim
            se o = array[inicio]
                devolve verdadeiro
            senão se o > array[parent(inicio)] e o < array[inicio]
                numNodos = numNodos + 1
            inicio = inicio +1
        se numNodos = nodos
            devolve falso
        nodos = nodos * 2
}
```



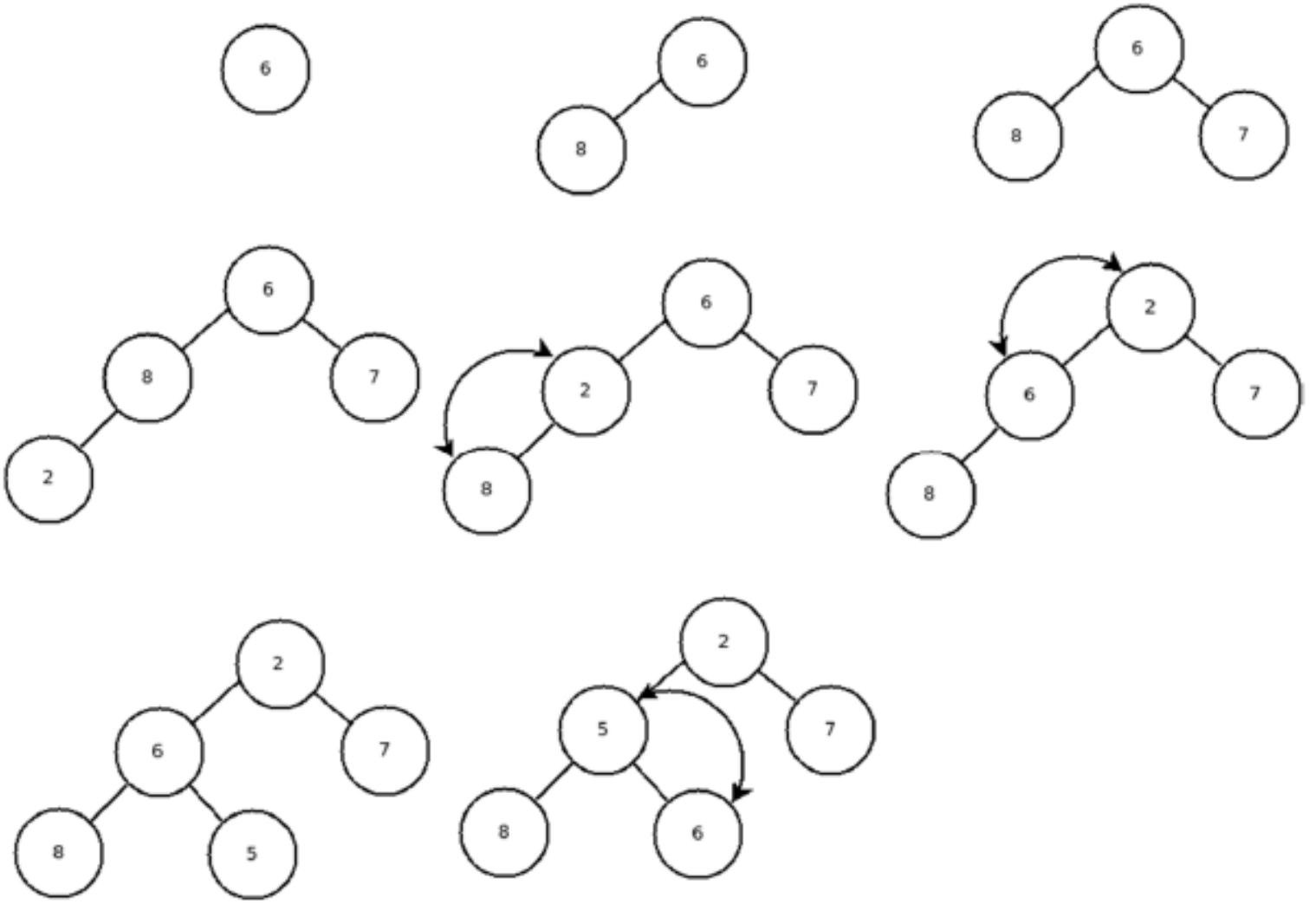
Exemplos: Heap Sort

Algortimo de ordenação com recurso a um heap:

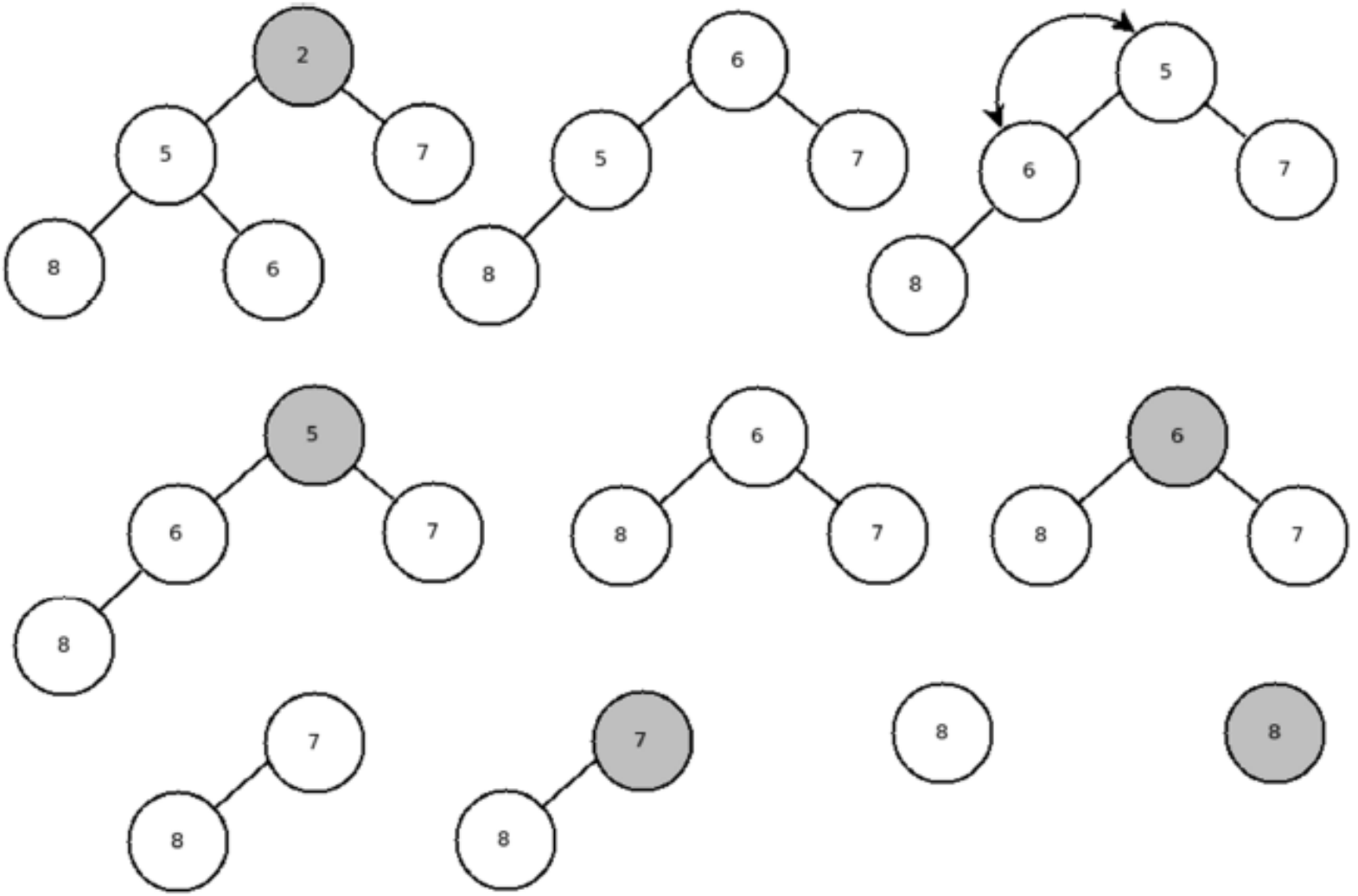
1. Introduzir os elementos num heap
2. Retirar o primeiro elemento do heap

Exemplo com 6, 8, 7, 2, 5

Exemplos: Heap Sort (Inserção)



Exemplos: Heap Sort (Remoção)



Exemplos: Paginação de Memória

